

Full SaaS Build Plan for BusinessOS AI and a Telly Prime-like Video Platform

Executive summary

The uploaded BusinessOS AI brief defines an extremely broad “system-as-a-service” product: website builder and deployer, CRM, marketing automation, analytics, support, operations, team productivity, inventory, admin, developer APIs, and a cross-cutting AI layer. Treated literally, that is not a 60-day MVP; it is a multi-year product portfolio. The right interpretation is to build a **modular monolith with workers**, ship a **narrow commercial wedge first**, and reserve the rest for staged expansion. For BusinessOS AI, the best wedge is: **multi-tenant auth + onboarding + CRM + website/CMS + deployment + analytics + AI assistant + basic admin/billing**. For a Telly Prime-like product, the best wedge is: **video upload + asset library + transcoding + HLS/DASH playback + embed player + analytics**, with live streaming as a controlled beta, not a day-one commitment. That recommendation aligns with the scope in the BusinessOS brief and with the operational reality of modern frontend, backend, auth, storage, queueing, observability, and media infrastructures. ¹

For the web stack, the strongest free/open-source default is **Next.js + Tailwind CSS + shadcn/ui** on the frontend, **NestJS** for the main API, **PostgreSQL + Prisma** for system-of-record data, **Auth.js** for lightweight app auth or **Keycloak/ZITADEL** for enterprise identity, **BullMQ + Redis** for background jobs, **MinIO or Cloudflare R2** for object storage, **PostHog + Metabase/Matomo + Grafana LGTM** for analytics and observability, and **LiteLLM + Ollama/vLLM + Langfuse** for the AI layer. Those choices minimize lock-in while keeping a clean modern UI path and preserving an enterprise migration story. ²

For the video product, the recommended free/open-source emulation of managed platforms is: **tus/tusd** for resumable ingestion, **FFmpeg** for transcoding, **Shaka Packager** for HLS/DASH packaging, **MediaMTX** or **SRS** for straightforward streaming/router duties, **OvenMediaEngine** for sub-second live delivery where required, **Video.js** or **hls.js/Shaka Player** for playback, and **ClickHouse + PostHog + OpenTelemetry** for playback/QoE analytics. The major limitation is structural: there is **no truly free, open-source equivalent of a global video CDN plus commercial DRM licensing service**. For MVPs, the realistic substitute is signed playback tokens, signed URLs, region-aware origins, object storage, and selective CDN/caching, not full enterprise OTT parity. ³

The exact product name “**Telly Prime**” could not be verified in indexed primary sources during this research; exact-term searches surfaced unrelated **TallyPrime** accounting pages rather than a primary video platform. Accordingly, the analysis below treats “Telly Prime-like” according to your stated requirement: a SaaS for **video hosting, streaming, analytics, deployment, and live delivery**. ⁴

Product scope and module strategy

The BusinessOS AI brief is best understood as a **platform of platforms**. If built naively, its biggest risk is not engineering failure but **surface-area dilution**: too many half-complete modules, weak data quality, and no compelling first workflow. The correct scope cut is to define a common tenant layer and then choose the modules that compound user value in one motion: **capture demand, publish presence, track relationships, measure outcomes, and automate next actions**. That is why the recommended BusinessOS MVP is centered on **Landing/Marketing site + Auth/Tenancy + Onboarding + Dashboard/Analytics + CMS/Website Builder + Deployer + CRM + AI Assistant + Admin/Plan Management**. Marketing automation, support, finance, productivity, inventory, and marketplace/developer expansions should be layered after the control plane is stable. ⁵

A Telly Prime-like product is narrower in business semantics but deeper in infrastructure semantics. Its minimum credible product is not “video upload exists”; it is **upload reliability, processing reliability, manifest correctness, playback reliability, usage accounting, and analytics trustworthiness**. That makes the natural module set: **Asset ingestion, media processing, playback/origin, live streaming, analytics, player/embed, tenant/admin, API/webhooks, and billing/usage metering**. Live is the most operationally fragile module because it couples encoder behavior, protocol negotiation, transcoding capacity, origin correctness, and end-user network conditions. On a 60-day plan, **VOD-first with live beta** is the soundest path. ⁶

Recommended BusinessOS AI module cut

Priority	Module	Ship in MVP	Why
Must-have	Tenancy, auth, roles, onboarding	Yes	Everything else depends on workspace identity and provisioning.
Must-have	CRM core	Yes	Gives BusinessOS a hard operational center and recurring daily use.
Must-have	Website/CMS + forms	Yes	Lets the platform create and capture demand from day one.
Must-have	Deployer + domains + preview/rollback	Yes	Creates a concrete “publish value” moment.
Must-have	Dashboard + attribution + AI summaries	Yes	Turns collected data into action.
Should-have	Basic support inbox/widget	Later in MVP or immediately after	Strong retention feature, but not required to prove core value.
Should-have	Email automation, sequences, referral/affiliate	Post-MVP	Valuable, but can initially piggyback on events + queues.

Priority	Module	Ship in MVP	Why
Later	Finance, inventory, orders, vendor management	V2	High breadth; better after core platform usage is proven.
Later	Plugin marketplace and public SDKs	V2/V3	Important strategically, premature operationally.

Recommended Telly-like module cut

Priority	Module	Ship in MVP	Why
Must-have	Uploads, asset library, processing jobs	Yes	Without this, there is no product.
Must-have	HLS/DASH manifests, renditions, playback tokens	Yes	Playback trust is the product's credibility.
Must-have	Embed player + player config	Yes	Makes the platform usable in customer apps/sites.
Must-have	Playback event ingestion + QoE analytics	Yes	Differentiates commodity file hosting from video SaaS.
Should-have	Live inputs and recording	Beta only	Useful commercially, but the biggest reliability risk.
Later	Simulcast/restream, DVR polish, advanced ABR ladders	V2	Nice expansion after stable ingest/playback.
Later	DRM, server-side ad insertion, advanced entitlements	Enterprise	Expensive and not truly available in a fully free/open-source stack.

Recommended free and open-source stack

The stack below is opinionated on purpose. The goal is not to maximize optionality; it is to minimize architectural regret during the first six to twelve months.

Primary stack and alternatives by layer

Layer	BusinessOS AI primary choice	Telly-like primary choice	Good free/ open-source alternatives	Why this is the best default	Primary sources
Frontend shell	Next.js + Tailwind + shadcn/ui	Next.js + Tailwind + shadcn/ui	Plain React on Cloudflare Pages; Strapi/ Directus/ Payload for content-heavy frontends	Next.js is explicitly full-stack; Tailwind is utility-first; shadcn/ui gives accessible, open-code components and a clean modern SaaS admin aesthetic.	Next.js/ Tailwind/ shadcn docs ⁷
Backend/API	NestJS modular monolith	NestJS control plane + separate media workers	Laravel if the team is PHP-heavy	NestJS is strongly suited to scalable Node/ TypeScript apps and has a clean eventual path to microservices.	NestJS docs ⁸
Transaction DB	PostgreSQL + Prisma	PostgreSQL + Prisma	Supabase-managed Postgres; Neon serverless Postgres	PostgreSQL is mature and supports row security; Prisma keeps schema and application types aligned.	PostgreSQL + Prisma docs ⁹
Tenancy pattern	Pool first, bridge by plan	Bridge first for control plane; silo/high-isolation later	Neon database-per-tenant for higher tiers	AWS's pool/ bridge/silo framing maps cleanly to SaaS plan tiers; Neon is unusually strong for database-per-tenant models.	AWS SaaS Lens + Neon multitenancy ¹⁰

Layer	BusinessOS AI primary choice	Telly-like primary choice	Good free/ open-source alternatives	Why this is the best default	Primary sources
Auth	Auth.js for lightweight app auth; Keycloak for enterprise SSO	Same	ZITADEL; Supabase Auth when using Supabase heavily	Auth.js is runtime-agnostic and secure with encrypted JWTs by default; Keycloak adds OIDC/SAML/ LDAP and enterprise federation.	Auth.js + Keycloak + ZITADEL docs ¹¹
Object storage	MinIO self-host or Cloudflare R2	MinIO for self-hosted media origin or R2 for cheap object delivery	Supabase Storage	MinIO gives S3 compatibility, OIDC, bucket quotas, notifications, audit telemetry, and multi-tenancy; R2 reduces egress pain and has a real free tier.	MinIO + R2 docs ¹²
Queueing	BullMQ + Redis	BullMQ + Redis for ingest/encode jobs	Upstash Redis free tier; RabbitMQ/ NATS later	BullMQ is a robust Redis-backed queue with queue, worker, flow, and event primitives; perfect for SaaS workflows and media jobs.	BullMQ docs ¹³
Workflow automation	n8n	n8n only for business-side automations, not media-critical path	Custom BullMQ orchestrations	n8n is strong for no-code/ low-code business automations but should not own the latency-critical media path.	n8n docs ¹⁴

Layer	BusinessOS AI primary choice	Telly-like primary choice	Good free/ open-source alternatives	Why this is the best default	Primary sources
Transactional email	Resend or Brevo	Resend or Brevo	SMTP relay; Mautic for campaign mail	Managed APIs reduce deliverability pain early; Mautic covers self-hosted campaign automation.	Resend + Brevo + Mautic docs ¹⁵
Product analytics	PostHog	PostHog for product events + ClickHouse for media/QoE aggregations	Matomo for privacy-sensitive analytics	PostHog is open-source/ self-hostable and covers product analytics, replay, feature flags, and APIs; ClickHouse handles high-volume event analytics.	PostHog + ClickHouse + Matomo docs ¹⁶
BI / dashboards	Metabase	Metabase or Grafana over ClickHouse/ Postgres	Matomo reports for web-only use cases	Metabase is simple, open-source, and easy to embed; Grafana is stronger for operational telemetry.	Metabase + Grafana docs ¹⁷
AI gateway / models	LiteLLM + Ollama locally; vLLM when GPU inference matters	Same	Direct provider calls only when necessary	LiteLLM normalizes APIs across providers and models; Ollama is easiest local model runtime; vLLM is the scalable backend when throughput matters.	LiteLLM + Ollama + vLLM docs ¹⁸

Layer	BusinessOS AI primary choice	Telly-like primary choice	Good free/ open-source alternatives	Why this is the best default	Primary sources
AI observability/ evals	Langfuse	Langfuse	PostHog LLM analytics supplement	Langfuse is purpose-built for LLM traces, evaluations, and debugging.	Langfuse docs ¹⁹
Deployment control plane	Coolify	Coolify	Dokku	Both are viable; Coolify is better as a modern self-hosted PaaS, Dokku is leaner on a single server.	Coolify + Dokku docs ²⁰
Frontend hosting	Cloudflare Pages first; Vercel Hobby acceptable for UI-only	Cloudflare Pages for dashboard/ docs/player shells	Self-host on Coolify if desired	Pages is available on all plans and gives global edge delivery; Vercel Hobby is a fast zero-cost starter for web apps.	Cloudflare Pages + Vercel pricing/docs ²¹
Monitoring and tracing	OpenTelemetry + Prometheus + Loki + Tempo + Grafana	Same	Sentry or GlitchTip for app errors	OTel keeps instrumentation portable; Loki/ Tempo/ Prometheus/ Grafana form an OSS observability stack; GlitchTip is a good open-source Sentry-compatible error tracker.	OTel + Prometheus + Loki + Tempo + GlitchTip docs ²²

Layer	BusinessOS AI primary choice	Telly-like primary choice	Good free/ open-source alternatives	Why this is the best default	Primary sources
Design and PM	Figma Starter + GitHub Projects	Same	Plane CE; Taiga	Figma Starter is free; GitHub Projects integrates naturally with repos; Plane and Taiga are self-hostable PM upgrades.	Figma + GitHub + Plane + Taiga docs ²³
Support/ helpdesk	Chatwoot	Chatwoot	Zammad; FreeScout	Chatwoot is the best omnichannel open-source support default; Zammad is strong for classic helpdesk; FreeScout is very light.	Chatwoot + Zammad + FreeScout docs ²⁴
CMS/content admin	Payload or Directus if content teams need schema-driven admin	Payload/Directus for docs/ marketing, not media control plane	Strapi	Use only if your content surface grows beyond simple page builder/CMS needs.	Payload + Directus + Strapi docs ²⁵
Commerce/ ops modules	Medusa for shop-like order flows; Akaunting for lightweight accounting	Usually not needed unless selling creator storefronts		BusinessOS can accelerate V2 commerce with existing OSS primitives instead of re-inventing catalog/order logic.	Medusa + Akaunting docs ²⁶

Layer	BusinessOS AI primary choice	Telly-like primary choice	Good free/ open-source alternatives	Why this is the best default	Primary sources
Video ingestion	N/A	tus/tusd	direct multipart uploads	tus gives resumable, reliable uploads; tusd is the official reference server.	tus docs ²⁷
Video processing	N/A	FFmpeg + Shaka Packager	Livepeer as optional managed/ open benchmark	FFmpeg handles encode/ transcode; Shaka Packager handles HLS/ DASH packaging and encryption but not transcoding.	FFmpeg + Shaka Packager docs ²⁸
Live streaming	N/A	MediaMTX or SRS for general use; OvenMediaEngine for sub-second	Livepeer for managed/ open hybrid	OME is ideal when WebRTC/ LL-HLS latency matters; SRS and MediaMTX are easier defaults.	MediaMTX + SRS + OME docs ²⁹
Video player	N/A	Video.js + hls.js or Shaka Player		Video.js is a mature open-source web player; hls.js is a proven HLS client; Shaka Player is strong where DASH matters.	Video.js + hls.js + Shaka Player docs ³⁰

Free-tier and hosting tradeoff reality check

A pure “no-cash” MVP is possible for **BusinessOS AI pilot traffic**, but only partially possible for a **video platform**. The hidden reason is bandwidth and media processing, not the application code itself. ³¹

Service	Official free or starter signal	Best use in MVP	Main gotcha
Cloudflare Pages	Available on all plans.	Marketing site, dashboard shell, docs, player/embeds.	Great for edge web delivery, not a NestJS replacement by itself. ³²
Vercel Hobby	"Free forever."	Fast frontend iteration if the team prefers Vercel workflows.	Usage and team controls become the constraint, not initial cost. ³³
Supabase Free	Unlimited API requests, 50,000 MAUs, 500 MB database, 1 GB file storage, 5 GB egress.	BusinessOS pilot where DB/auth/storage are bundled.	Free projects hit read-only mode once database size exceeds the quota. ³⁴
Neon Free	100 projects, 100 CU-hours monthly per project, 0.5 GB storage per project, autoscaling, scale-to-zero, 60K MAUs via Neon Auth.	Excellent for multi-tenant experiments and DB-per-tenant higher-end plans.	Great economics for bursty traffic, but not a free substitute for high sustained load. ³⁵
Upstash Redis Free	500K commands/month, 256 MB, one free DB.	Dev/staging queues, rate limiting, small automations.	It will cap out quickly for noisy media pipelines. ³⁶
Cloudflare R2 Free tier	10 GB-month storage, 1M Class A ops, 10M Class B ops monthly, no direct egress fees from R2.	Static/media storage for early pilots, especially BusinessOS assets and small VOD pilots.	Reads dominate video costs fast; cheap is not the same as free at scale. ³⁷
PostHog self-host	Open-source, MIT, self-hostable with Docker Compose.	Product analytics without immediate SaaS spend.	Self-hosting is intentionally "hobbyist" unless you own the ops burden. ³⁸
Coolify	Open-source PaaS; recommends fresh server with at least 2 cores, 2 GB RAM, 30 GB free space.	Best self-host control plane if you want "one place to run the stack."	The software is free; the server is not. ³⁹

Bottom line on hosting: for BusinessOS AI, a near-\$0 pilot stack can be **Cloudflare Pages + Neon or Supabase Free + Upstash Free + R2 free tier + self-hosted PostHog/Metabase later**. For Telly-like products, free tiers are enough to validate the control plane, but **not** enough to validate sustained playback economics; even cheap object reads and delivery minutes compound quickly. Managed video services make this visible by billing storage and delivery minutes explicitly, which is a strong hint that your own pricing should meter the same units. ⁴⁰

Architecture, tenancy, data model, and API design

The best starting architecture for **both** products is a **modular monolith plus workers**. For BusinessOS AI, that means one application repository and one main API process, organized into modules with clear boundaries and an outbox/event pattern. For Telly-like products, it means one control plane monolith plus **separate stateless worker pools** for ingest, transcode, packaging, thumbnailing, and analytics enrichment. Do **not** split either product into many microservices before you have operational evidence. Even the AWS SaaS guidance emphasizes choosing from pool, bridge, and silo models based on isolation and operational needs, not ideology. ⁴¹

The tenancy recommendation is different by product. BusinessOS AI should begin with **pool + bridge**: shared app tier, shared database, strict `tenant_id` scoping everywhere, row-level security where practical, and then separate databases or schemas for premium/regulated tenants. Telly-like platforms should start with a **bridge** model: shared control plane, shared analytics infrastructure, but storage buckets, playback keys, rate limits, and optionally separate databases for higher tiers. That split keeps the control plane efficient while creating a path to enterprise isolation later. PostgreSQL row security, Supabase's RLS guidance, and Neon's database-per-tenant design all support this progression. ⁴²

```
flowchart LR
    subgraph Users
        A[Business users]
        B[Site visitors / leads]
        C[Video admins]
        D[Video viewers]
    end

    subgraph Edge
        E[Next.js apps on Pages/Vercel]
        F[Custom domains + SSL + CDN]
    end

    subgraph ControlPlane[Control plane]
        G[NestJS modular monolith]
        H[Auth.js / Keycloak]
        I[PostgreSQL + Prisma]
        J[Redis + BullMQ]
        K[Webhooks + API keys]
    end

    subgraph BusinessOS[BusinessOS AI modules]
        L[CRM + Contacts + Deals]
        M[Website/CMS + Forms]
        N[Deployments + Domains + Releases]
        O[Dashboards + Attribution]
        P[AI assistant + workflows]
        Q[Admin + Plans + Usage]
    end
```

```

end

subgraph VideoPlane[Telly-like media plane]
  R[tusd uploads]
  S[FFmpeg transcode workers]
  T[Shaka Packager]
  U[MinIO / R2 origin storage]
  V[MediaMTX / SRS / OME live]
  W[Player + tokens + embeds]
  X[Playback events + QoE]
  Y[ClickHouse + PostHog]
end

subgraph Observability
  Z[OpenTelemetry]
  AA[Prometheus / Loki / Tempo / Grafana]
  AB[GlitchTip / Sentry-style errors]
  AC[Langfuse for AI traces]
end

A --> E
B --> E
C --> E
D --> W
E --> F
E --> G
F --> E
G --> H
G --> I
G --> J
G --> K

G --> L
G --> M
G --> N
G --> O
G --> P
G --> Q

G --> R
R --> U
G --> J
J --> S
S --> T
T --> U
G --> V
U --> W
W --> X

```

X --> Y

G --> Z

Z --> AA

G --> AB

P --> AC

Key data model highlights

The tables below are not exhaustive schemas. They are the **minimum durable entities** that keep the products coherent.

BusinessOS AI core tables

Domain	Key tables	Must-have fields
Tenant and identity	tenants, users, memberships, roles, permission_grants	id, slug, plan_id, region, status, user_id, tenant_id, role_key, invited_by, mfa_enabled, last_login_at
Commercial control plane	plans, subscriptions, feature_entitlements, usage_meters, usage_ledger	plan_code, billing_interval, seat_limit, storage_limit_mb, ai_credit_limit, meter_key, period_start, period_end, consumed, hard_limit, soft_limit
Onboarding	onboarding_runs, imports, templates, template_installs	industry, wizard_state, source_type, mapping_json, template_type, installed_resources_json
CRM	contacts, companies, leads, pipelines, pipeline_stages, deals, activities, notes, tags, entity_tags	tenant_id, owner_user_id, email, phone, lifecycle_stage, source, score, pipeline_id, stage_id, amount, currency, last_activity_at
Websites/ CMS	sites, pages, page_versions, blocks, themes, assets, forms, form_fields, form_submissions, locales	site_id, route_path, status, published_version_id, schema_json, theme_tokens_json, storage_key, submission_ip, utm_json, locale_code

Domain	Key tables	Must-have fields
Deployment	deployments , release_artifacts , domains , certificates , preview_environments	site_id , commit_sha , build_status , deploy_status , artifact_uri , domain , verification_status , ssl_status , expires_at
Analytics	events , sessions , funnels , funnel_steps , ab_tests , ab_variants , attribution_touchpoints	event_name , anonymous_id , user_id , utm_source , referrer , session_id , variant_key , goal_event , touch_type
Marketing automation	campaigns , segments , automation_flows , automation_runs , messages , message_deliveries	channel , audience_rule_json , trigger_json , status , scheduled_at , provider_message_id , delivery_state
Support	inboxes , conversations , messages , tickets , kb_articles , kb_categories	channel_type , external_thread_id , assigned_to , priority , sla_due_at , article_slug , visibility
Finance and ops	invoices , invoice_items , payments , expenses , vendors	invoice_number , due_date , customer_ref , subtotal , tax_total , payment_provider , payment_status , expense_category
Productivity	projects , tasks , task_comments , calendar_events , reminders	assignee_id , status , due_at , estimate_minutes , event_type , remind_at
Catalog/ business modules	products , inventory_items , orders , order_items , coupons , reviews	sku , qty_available , reorder_level , order_status , discount_type , rating , source_channel
AI/ developer/ admin	knowledge_documents , document_chunks , ai_runs , ai_feedback , api_keys , webhook_endpoints , audit_logs	embedding_status , prompt_template_key , model_key , cost_estimate , latency_ms , scope_json , event_mask , actor_id , entity_type , before_json , after_json

Telly-like core tables

Domain	Key tables	Must-have fields
Tenant and plans	tenants, users, memberships, subscriptions, usage_meters	tenant_id, plan_code, storage_bytes_used, delivery_minutes_used, live_channel_count, overage_policy
Asset ingestion	assets, uploads, upload_parts, ingest_jobs	asset_id, source_filename, source_mime, tus_upload_id, byte_size, checksum, ingest_state, failure_reason
Processing	renditions, manifests, thumbnails, captions, drm_profiles	codec, bitrate, width, height, duration_ms, manifest_type, playlist_uri, thumbnail_sprite_uri, language, is_default
Playback security	playback_policies, signed_tokens, domains, players	policy_type, allow_origin, expires_at, token_jti, player_theme_json, watermark_enabled
Live	live_inputs, live_sessions, simulcast_targets, recordings	protocol, stream_key_hash, ingest_url, session_state, started_at, stopped_at, recording_asset_id
Analytics	viewer_sessions, playback_events, qoe_rollups, geo_rollups, device_rollups	viewer_session_id, asset_id, startup_time_ms, rebuffer_count, rebuffer_duration_ms, watch_time_ms, country_code, device_family, player_version
Platform/dev/admin	api_keys, webhook_endpoints, audit_logs, abuse_cases	scope_json, target_url, retry_policy, event_type, actor_id, abuse_reason, blocked_until

Recommended API surface by module

These are **recommended product APIs**, not vendor APIs.

Product	Module	Recommended API surface
BusinessOS AI	Identity and tenancy	POST /auth/register, POST /auth/login, POST /auth/oauth/callback, GET /me, POST /tenants, POST /tenants/:id/invitations, PATCH /memberships/:id

Product	Module	Recommended API surface
BusinessOS AI	Onboarding	POST /onboarding/start, POST /onboarding/import, POST /templates/:id/install, GET /onboarding/:runId/status
BusinessOS AI	CRM	GET/POST /contacts, GET/POST /companies, GET/POST /deals, PATCH /deals/:id/stage, POST /activities, POST /segments/preview
BusinessOS AI	Website/CMS	GET/POST /sites, GET/POST /sites/:id/pages, POST /pages/:id/versions, POST /sites/:id/forms, POST /forms/:id/submissions, POST /assets/upload-init
BusinessOS AI	Deployments	POST /sites/:id/deploy, GET /deployments/:id, POST /deployments/:id/rollback, POST /domains/connect, POST /domains/verify
BusinessOS AI	Analytics/marketing	POST /events, GET /dashboards/overview, GET /funnels/:id/report, POST /campaigns, POST /automations, POST /attribution/recompute
BusinessOS AI	AI	POST /ai/chat, POST /ai/generate-site, POST /ai/generate-content, POST /ai/score-lead, POST /ai/insights/run
BusinessOS AI	Support/admin/dev	POST /inboxes, POST /conversations, POST /tickets, POST /api-keys, POST /webhooks, GET /admin/tenants, GET /admin/usage
Telly-like	Assets/ingest	POST /uploads/init, PATCH /uploads/:id/chunk, POST /assets, GET /assets/:id, POST /assets/:id/reprocess
Telly-like	Encoding/packaging	GET /jobs/:id, GET /assets/:id/renditions, GET /assets/:id/manifests, POST /assets/:id/captions
Telly-like	Playback/security	POST /assets/:id/playback-token, GET /playback/:assetId/master.m3u8, GET /players/:id/embed, POST /domains, POST /playback/policies
Telly-like	Live	POST /live/inputs, GET /live/inputs/:id, POST /live/inputs/:id/start, POST /live/inputs/:id/stop, GET /recordings/:id
Telly-like	Analytics/admin	POST /analytics/playback-events, GET /analytics/assets/:id, GET /analytics/qoe, POST /webhooks, GET /usage, GET /audit-logs

Security, compliance, and AI safety controls

The baseline security standard should be **OWASP ASVS-aligned**, with specific attention to the current OWASP Top 10 and the OWASP GenAI risk taxonomy for AI-enabled systems. That means secure defaults in auth flows, tenant isolation, secrets management, input validation, auditability, supply chain hygiene, and defensive output handling. For AI features, the two biggest practical risks are **prompt injection** and **sensitive information disclosure**, so the AI layer must be treated as a semi-trusted subsystem—not an omnipotent administrator. ⁴³

At the application layer, use **Auth.js** for simple app-native auth or **Keycloak/ZITADEL** for enterprise SSO, MFA, and identity-provider federation. At the database layer, use **tenant-scoped queries everywhere** and enable PostgreSQL row-level security for the most sensitive shared tables. At the infrastructure layer, enforce TLS end to end, prefer “Full (strict)” style origin verification, and publish explicit security headers including CSP, X-Frame-Options, X-Content-Type-Options, Permissions-Policy, and Referrer-Policy. Those are table stakes, not premium features. ⁴⁴

For compliance, do not promise certifications you do not yet hold. In the first year, the realistic target is to be **compliance-ready by architecture**: data classification, consent records, retention controls, export/delete flows, DPA templates, region-aware deployment choices, and per-tenant audit trails. For AI governance, align process design with the NIST AI RMF and GenAI profile: identify risks, document model routing and prompt policies, test with evaluations and red-teaming, and maintain human override for material business actions. ⁴⁵

Security control matrix

Control area	BusinessOS AI	Telly-like product	Why it matters
Tenant isolation	<code>tenant_id</code> on all pooled entities; RLS on sensitive tables; bridge/silo for premium tenants	Shared control plane, per-tenant storage prefixes/buckets, isolated playback keys, optional separate DBs for enterprise	Prevents cross-tenant leakage and noisy-neighbor failures. ⁴⁶
Identity	Passwordless/OAuth for SMB; SSO/SAML/OIDC + MFA for business/enterprise	Same	Identity posture scales with contract size. ⁴⁷
Session/token safety	Short-lived sessions, rotating refresh tokens, encrypted JWTs, device/session revocation	Same + signed playback tokens and signed URLs	Limits replay and token leakage blast radius. ⁴⁸
Secrets and keys	Central secret storage; separate envs; rotate provider keys quarterly	Separate signing keys for playback and uploader credentials	AI and media keys are attractive exfiltration targets.

Control area	BusinessOS AI	Telly-like product	Why it matters
Web security	CSP, strict headers, CSRF protections, rate limiting, webhook signature verification	Same + CORS/origin restrictions on player and ingest	Protects browser surfaces and public APIs. ⁴⁹
Observability and audit	OTel traces, audit logs, admin event history, webhook attempt logs	Same + per-stream/live session audit trails	Needed for incident response and enterprise trust. ⁵⁰
AI safety	Prompt template versioning, tool allowlists, retrieval allowlists, PII redaction, output validation, human approval for destructive actions	Same + never let model directly expose storage keys/origin creds	Prompt injection is inevitable; blast-radius design is the defense. ⁵¹
Video abuse controls	N/A	Signed playback, geo/time/IP restrictions, expiring embeds, watermarking later, abuse throttles	Prevents hotlinking, piracy-style abuse, and origin drains.
Secure development process	ASVS checklist, dependency review, SBOM where possible, review gates for auth/billing/media	Same	You will grow faster than your memory; process creates repeatability. ⁵²

MVP roadmap, effort, and pricing

Because team size and budget are unspecified, the effort below is expressed in **person-weeks**, not elapsed headcount-normalized promises. A 30–60 day plan assumes concentrated execution by a small product team with senior engineering judgment, strong reuse of existing open-source components, and ruthless scope control.

```

gantt
  title 60-day MVP timeline
  dateFormat YYYY-MM-DD
  axisFormat %d %b

  section BusinessOS AI
  Tenant/auth/onboarding      :a1, 2026-05-04, 10d
  CRM core + data model       :a2, after a1, 10d
  Website/CMS + forms         :a3, 2026-05-10, 16d
  Deployments + domains + preview :a4, 2026-05-18, 14d
  Dashboard + attribution + AI chat :a5, 2026-05-22, 14d
  Admin/plans/usage + hardening :a6, 2026-06-05, 10d
  Support inbox + polish      :a7, 2026-06-12, 14d

```

section Telly-like	
Tenant/auth/control plane	:b1, 2026-05-04, 10d
Uploads + asset library	:b2, after b1, 8d
Transcode + packaging pipeline	:b3, 2026-05-14, 16d
Playback/player/tokens	:b4, 2026-05-24, 12d
Event ingestion + QoE analytics	:b5, 2026-05-28, 14d
Live beta + recordings	:b6, 2026-06-10, 14d
Billing/usage/admin + hardening	:b7, 2026-06-18, 10d

BusinessOS AI prioritized MVP roadmap

Milestone	Deliverables	Estimated effort
Foundation	Tenant model, roles, auth, invitations, onboarding wizard, plan/entitlement skeleton, design system, repo and CI baseline	5–6 person-weeks
Revenue core	Contacts, companies, deals, pipeline stages, notes, activities, CSV import, dashboard cards	5–6 person-weeks
Presence core	Sites, pages, page versions, theme tokens, forms, asset upload, SEO fields, templates	5–6 person-weeks
Publish core	Build/deploy pipeline, release history, preview environments, custom domains, SSL status, rollback, usage accounting	4–5 person-weeks
Intelligence core	Unified events, attribution, dashboard widgets, AI assistant with tenant-scoped retrieval, AI-written page/content helpers	4–5 person-weeks
Commercial/admin core	Admin panel, plan assignment, feature flags, usage meters, audit logs, billing placeholders, hardening and QA	4–5 person-weeks
Stretch in days 45–60	Support inbox/widget, basic campaign sequences, richer onboarding templates	3–4 person-weeks

BusinessOS AI total MVP estimate: 26–33 person-weeks.

Telly-like prioritized MVP roadmap

Milestone	Deliverables	Estimated effort
Foundation	Tenant model, auth, API keys, plans, usage meters, asset metadata schema, admin shell	4–5 person-weeks
Reliable ingest	Resumable uploads via tusd, object storage layout, upload verification, checksums, asset library UI	4–5 person-weeks

Milestone	Deliverables	Estimated effort
Processing pipeline	Job orchestration, FFmpeg transcodes, ABR ladder presets, thumbnails, captions metadata, Shaka Packager HLS/DASH outputs	6–7 person-weeks
Playback and embed	Playback tokens, player presets, Video.js/hls.js embed, domain whitelisting, signed URLs, manifest/origin delivery	4–5 person-weeks
Analytics	Playback event SDK, viewer sessions, startup/rebuffer/watch-time metrics, ClickHouse rollups, dashboards	4–5 person-weeks
Live beta	One or two live input types, recording-to-VOD, stream health, simple restream/simulcast hooks, incident logging	5–6 person-weeks
Commercial/admin	Usage reporting, invoices or billable usage exports, webhook delivery, audit logs, hardening and QA	4–5 person-weeks

Telly-like total MVP estimate: 31–38 person-weeks for VOD + controlled live beta.

Feature gating and pricing strategy

The pricing model should follow **cost causality**. For BusinessOS AI, the expensive units are seats, contacts, automations, storage, domains, background-job execution, and AI usage. For Telly-like products, the expensive units are storage, processing minutes, delivery minutes, concurrent live channels, and analytics retention. Managed infrastructure pricing already reflects this logic: serverless databases charge on active compute/storage, object stores charge on storage and operations, and managed video platforms charge on stored and delivered video. Your pricing should mirror the same hard-cost vectors. ⁵³

Recommended BusinessOS AI plans

Plan	Price posture	Gate by
Free	Product-led entry	1 workspace, 1 user, 1 site, low contact cap, basic CRM, no custom domain, tiny AI allowance
Starter	Low-friction SMB	3–5 seats, 1–3 domains, moderate contacts, simple analytics, basic forms, preview deploys
Pro	Main monetization tier	More seats, automations, AI content generation, custom dashboards, branded forms/sites, API keys
Business	Teams and agencies	SSO, audit logs, role granularity, more workspaces/clients, higher job/storage caps, priority support
Enterprise	Contract tier	Bridge/silo isolation, custom region, DPA, private AI routing, advanced governance, optional dedicated DB

Recommended Telly-like plans

Plan	Price posture	Gate by
Dev / Free	Builder adoption	Very small storage and delivery allowance, watermark or branding, one player preset, minimal analytics retention
Creator	Individual businesses	More storage, player branding controls, signed playback, basic captions/thumbnails, limited embed domains
Pro	SaaS/media teams	Processing presets, QoE analytics, API/webhooks, multiple players, higher delivery/storage caps
Business	Production teams	Live channels, recording controls, longer retention, priority transcoding, SSO, audit logs
Enterprise	Media/compliance tier	Dedicated regions/origins, premium isolation, advanced entitlement, DRM integrations, SLA, procurement support

Migration path and side-by-side analysis

The migration path should be staged by **scaling boundary**, not by module vanity. For BusinessOS AI, split services only after you can prove that one module has fundamentally different availability or throughput needs. The first likely breakouts are **deployment/build workers**, **event ingestion/analytics**, **AI gateway**, and **support/messaging adapters**. For Telly-like products, split earlier where the scaling physics are already known: **ingest**, **transcoding**, **packaging**, **playback token service**, and **analytics ingestion** should be separable even if the public API remains unified. ⁵⁴

Enterprise features should also follow a predictable ladder. First come governance features—SSO, audit logs, retention policy, data export/delete, custom domains, and webhooks. Then come isolation features—bridge/silo tenancy, dedicated databases, VPC/private networking, IP allow lists, and region pinning. Only after that should you consider heavyweight options such as marketplace ecosystems, white-label partner control planes, private AI inference, or advanced video rights controls. That ordering preserves commercial leverage while avoiding architecture overreach. ⁵⁵

BusinessOS AI versus Telly-like product

Dimension	BusinessOS AI	Telly-like product
Core value proposition	Broad operational OS for SMBs/agencies	Vertical infrastructure product for video delivery and insight
Best first wedge	CRM + sites + deploy + analytics + AI assistant	VOD pipeline + embeds + analytics
Engineering risk shape	Breadth and integration sprawl	Bandwidth, latency, processing, and reliability

Dimension	BusinessOS AI	Telly-like product
Operational cost profile	Mostly predictable and low at pilot scale	Highly variable because storage, reads, and delivery compound
Free-tier suitability	Good for a pilot and early paying beta	Good for control-plane validation, poor for meaningful traffic validation
Data model complexity	Broad but conventional SaaS entities	Smaller control plane, much heavier event/telemetry volumes
Hardest subsystem	Cohesive UX across modules	Media processing and playback correctness
Enterprise upsell path	Governance, analytics depth, AI, multi-workspace/agency support	Isolation, live reliability, analytics depth, rights controls, SSO/SLA
Approximate MVP effort	26–33 person-weeks	31–38 person-weeks
Merits	Strong retention if modules share data well; large expansion surface	Clearer initial buyer story; deeper moat if playback/QoE are excellent
Risks	Easy to become a fragmented “suite of half-products”	Easy to burn money on delivery before pricing discipline exists
My recommendation	Build only the narrow wedge first; avoid finance/inventory in MVP	Build VOD-first; treat live as beta; meter delivery from day one

Open questions and limitations

The biggest research limitation is naming, not architecture: the exact **“Telly Prime”** product could not be confirmed from primary indexed sources, so the comparison uses the feature profile you specified rather than a verified vendor specification. Searches for the term surfaced unrelated TallyPrime accounting material instead of a primary video SaaS page. ⁴

The second limitation is platform economics: fully free/open-source tools can credibly emulate much of a commercial video platform’s pipeline, but they do **not** fully replace a global CDN, commercial DRM/rights stack, or the operational convenience of an all-in-one managed video API. If you need true enterprise OTT behavior, plan for managed infrastructure or premium self-hosting spend later, even if the MVP begins on open tools. ⁵⁶

¹ ² ⁵ ⁷ Next.js Docs

https://nextjs.org/docs?utm_source=chatgpt.com

³ ⁶ ²⁷ <https://tus.io/>

<https://tus.io/>

⁴ Tally Prime Education, Tally Education

https://tallyprimeeducation.com/?utm_source=chatgpt.com

- 8 Documentation | NestJS - A progressive Node.js framework
https://docs.nestjs.com/?utm_source=chatgpt.com
- 9 <https://www.postgresql.org/docs/>
<https://www.postgresql.org/docs/>
- 10 41 <https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/silo-pool-and-bridge-models.html>
<https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/silo-pool-and-bridge-models.html>
- 11 47 <https://authjs.dev/getting-started>
<https://authjs.dev/getting-started>
- 12 <https://min.io/docs/minio/linux/index.html>
<https://min.io/docs/minio/linux/index.html>
- 13 <https://docs.bullmq.io/>
<https://docs.bullmq.io/>
- 14 <https://docs.n8n.io/>
<https://docs.n8n.io/>
- 15 <https://resend.com/docs/introduction>
<https://resend.com/docs/introduction>
- 16 <https://posthog.com/docs>
<https://posthog.com/docs>
- 17 <https://www.metabase.com/docs/latest/>
<https://www.metabase.com/docs/latest/>
- 18 <https://docs.litellm.ai/docs/>
<https://docs.litellm.ai/docs/>
- 19 <https://langfuse.com/docs>
<https://langfuse.com/docs>
- 20 39 <https://coolify.io/docs/>
<https://coolify.io/docs/>
- 21 32 <https://developers.cloudflare.com/pages/>
<https://developers.cloudflare.com/pages/>
- 22 50 <https://opentelemetry.io/>
<https://opentelemetry.io/>
- 23 <https://www.figma.com/pricing/>
<https://www.figma.com/pricing/>
- 24 <https://www.chatwoot.com/>
<https://www.chatwoot.com/>
- 25 <https://payloadcms.com/docs/getting-started/what-is-payload>
<https://payloadcms.com/docs/getting-started/what-is-payload>
- 26 <https://docs.medusajs.com/learn>
<https://docs.medusajs.com/learn>

28 <https://ffmpeg.org/documentation.html>
<https://ffmpeg.org/documentation.html>

29 <https://mediamtx.org/docs/kickoff/install>
<https://mediamtx.org/docs/kickoff/install>

30 <https://videojs.com/>
<https://videojs.com/>

31 34 <https://supabase.com/pricing>
<https://supabase.com/pricing>

33 <https://vercel.com/pricing>
<https://vercel.com/pricing>

35 53 55 <https://neon.tech/pricing>
<https://neon.tech/pricing>

36 <https://upstash.com/docs/redis/howto/upgradedatabase>
<https://upstash.com/docs/redis/howto/upgradedatabase>

37 40 <https://developers.cloudflare.com/r2/pricing/>
<https://developers.cloudflare.com/r2/pricing/>

38 <https://posthog.com/docs/self-host>
<https://posthog.com/docs/self-host>

42 46 <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>
<https://www.postgresql.org/docs/current/ddl-rowsecurity.html>

43 52 <https://owasp.org/www-project-application-security-verification-standard/>
<https://owasp.org/www-project-application-security-verification-standard/>

44 48 <https://authjs.dev/reference/core/jwt>
<https://authjs.dev/reference/core/jwt>

45 <https://www.nist.gov/itl/ai-risk-management-framework>
<https://www.nist.gov/itl/ai-risk-management-framework>

49 <https://developers.cloudflare.com/pages/configuration/headers/>
<https://developers.cloudflare.com/pages/configuration/headers/>

51 <https://developers.openai.com/api/docs/guides/agent-builder-safety>
<https://developers.openai.com/api/docs/guides/agent-builder-safety>

54 [Microservices | NestJS - A progressive Node.js framework](https://docs.nestjs.com/microservices/basics?utm_source=chatgpt.com)
https://docs.nestjs.com/microservices/basics?utm_source=chatgpt.com

56 [Overview · Cloudflare Stream docs](https://developers.cloudflare.com/stream/?utm_source=chatgpt.com)
https://developers.cloudflare.com/stream/?utm_source=chatgpt.com